Multiplicación de números muy grandes

Un algoritmo clásico que aprendemos en el secundario para multiplicar números de n y m dígitos respectivamente es $\mathcal{O}(nm)$. Mientras trabajamos con unos pocos dígitos esto no es problema en general.

Ahora bien, cuando trabajamos con una enorme cantidad de dígitos, esto obviamente puede ser un problema. Es usual multiplicar números primos de **muchos** dígitos para aplicaciones de criptografía, por ejemplo.

Suponiendo que escribimos un número en base 2 (ya que para la computadora es fácil "ver" los números en binario), podemos decir que un número x cualquiera de n dígitos puede escribirse como: $x = x_1 2^{\frac{n}{2}} + x_0$. Por ejemplo, si $x = 11000110$, entonces $x_1 = 1100$ y $x_0 = 0110$. x_1 puede obtenerse de hacer $\frac{n}{2}$ *shifts-rights*, mientras que x_0 puede obtenerse de simplemente hacer un *and lógico* con la máscara de 0..01..1 ($\frac{n}{2}$ 0s seguidos de $\frac{n}{2}$ 1s).

Entonces para una multiplicación $x \cdot y = \left(x_1 2^{\frac{n}{2}} + x_0\right) \cdot y_1 2^{\frac{n}{2}} + y_0$ (suponemos ambos con misma cantidad de dígitos, si alguno tiene menos es agregar 0s a la izquierda).

Entonces:

$$x \cdot y = x_1 y_1 2^n + (x_1 y_0 + x_0 y_1) 2^{\frac{n}{2}} + x_0 y_0$$

Notar que esa multiplicación por potencias de 2 no es más que hacer *shift-lefts* y nada más (lo cual demanda tiempo lineal en cantidad de dígitos).

Planteando así, tendríamos 4 multiplicaciones a resolver, de la mitad de tamaño, con lo cual tendríamos una recurrencia $T(n) = 4T\left(\frac{n}{2}\right) + \mathcal{O}(n) \rightarrow T(n) = \mathcal{O}(n^2)$.

Karatsuba-Offman plantea como cambio recordar que $(x_1 + x_0)(y_1 + y_0) = x_1 y_1 + x_1 y_0 + x_0 y_1 + x_0 y_0$. Si calculamos $x_1 y_1$ y $x_0 y_0$ (que los necesitamos), podemos al resultado de la multiplicación restarle ambos, y nos quedaría $x_1 y_0 + x_0 y_1$. Con una sola multiplicación tendríamos 2 términos que nos faltan.

Código

```python
def multiplicacion_big_int(x, y):
    n = len(x)
    if n < MANEJABLE: # ej MANEJABLE = 32
        return x * y
    x1, x0 = separar(x)
    y1, y0 = separar(y)
    p = multiplicacion_big_int(x1 + x0, y1 + y0)
    x0y0 = multiplicacion_big_int(x0, y0)
    x1y1 = multiplicacion_big_int(x1, y1)
    restante = p - x0y0 - x1y1
    return x1y1 * 2**n + restante * 2**(n//2) + x0y0 # suponiendo que multiplicar por 2 es hacer un shift

def separar(num):
    n = len(num)
    num1 = num
    # Para obtener num0 podemos hacer un and logico, o irnos quedando con lo que se va cayendo
    # al ir obteniendo num1
    num0 = 0
    potencia2 = 1
    for i in range(n//2):
        num0 += (num1 % 2) * potencia2
        num1 /= 2
        potencia2 *= 2
    return num1, num0
```

Complejidad

La ecuación de recurrencia en este caso nos termina quedando:

$$T(n) = 3T\left(\frac{n}{2}\right) + \mathcal{O}(n)$$

Aplicando el teorema maestro: $A = 3; B = 2; C = 1 \rightarrow log_2 3 > 1 \rightarrow T(n) = \mathcal{O}\left(n^{\log_2 3}\right) \approx \mathcal{O}\left(n^{1.6}\right)$